

Sensor Controller Firmware

Firmware for an **AVR128DB32**-based temperature sensor board that exposes measurements over **Modbus RTU** on **RS485**.

This board currently supports:

- **8x NTC** temperature channels
- **2x DS18B20** 1-Wire sensors
- **1x K-type thermocouple** interface
- **RS485 / Modbus RTU** slave communications

This is a **Microchip MPLAB X** project using:

- **MPLAB X** embedded make project format
- **XC8** toolchain version **3.10**
- **AVR-Dx_DFP** pack version **2.7.321**
- **Atmel-ICE** project programmer configuration

The project metadata currently targets:

- Device: **AVR128DB32**
- Programmer interface: **UPDI**
- Programmer in project config: **AtmelIceTool**

Purpose

The firmware continuously samples the board sensors, stages the data into a Modbus register table, and responds to requests from an external master. It is structured as a simple foreground loop with a few timer/UART callbacks:

- the main loop handles **modbus_process()** and **poll_sensors()**
- **RTC PIT** provides a 1 Hz heartbeat / uptime tick
- **TCB0** closes Modbus frames using an inter-character timeout
- **TCB1** times DS18B20 conversion completion
- **TCB2** marks the thermocouple path ready to read

Project Status

This firmware is functional enough, but there are some important current-state notes that future development should understand up front:

- **DHT11/DHT22** support is present in the tree but disabled in the active polling path.
- The K-type path is currently treated as ready after the first **TCB2** callback and then read every loop pass.
- **NTC** acquisition is synchronous and relatively heavy because all enabled channels are sampled in the foreground loop.
- **DS18B20** is handled asynchronously with a shared conversion/read state machine.

- Register scaling is not fully uniform:
 - NTC values and most derived values are effectively stored as `degC x10`
 - DS18B20 values are stored as `degC x100`
 - K-type values are currently stored as `degC x100`
- The thermocouple conversion path currently adds a `+35` offset in code. Treat that as intentional current behavior until it is reviewed against hardware calibration requirements.

High-Level Firmware Architecture

The active entry points in `main.c` are:

- `_SYS_INIT()` at line 72
- `update_sensor_registers()` at line 109
- `poll_sensors()` at line 145
- `PIT()` at line 210
- `main()` at line 223

The active runtime sequence is:

1. `SYSTEM_Initialize()` configures the MCU using MCC-generated code.
2. `RTC_SetPITIsrCallback(PIT)` registers the 1 Hz uptime callback.
3. `_SYS_INIT()` initializes the register table, Modbus, K-type path, and DS18B20 path.
4. RS485 is forced into receive mode.
5. Global interrupts are enabled.
6. The foreground loop runs forever:
 - `modbus_process()`
 - `poll_sensors()`

Sensor Polling Summary

The polling path is controlled by `SREG_STATUS` and `SREG_SENSOR_ENABLE`.

- If `STATUS_POLL_ACTIVE` is clear, no active sensor polling occurs.
- If it is set:
 - enabled NTC channels are sampled
 - the thermocouple is read if its state machine is ready
 - both DS18B20 channels are read when the shared DS state reaches ready
 - the register table is refreshed from the latest sensor structs
 - derived temperatures such as chamber, superheat, subcooling, external, and evaporator are updated

Communications Summary

The board acts as a Modbus RTU slave on `UART1`.

Current communication behavior from `[functions/modbus.c]`:

- Slave address: 2
- Expected line format: RS485, 9600 baud, 8N1

- Supported function codes:
 - `0x03` Read Holding Registers
 - `0x04` Read Input Registers
 - `0x10` Write Multiple Registers

Frame reception is interrupt-driven:

- `UART1` RX callback appends bytes to `rx_buffer`
- `TCB0` timeout marks the frame complete
- `modbus_process()` validates address and CRC
- read/write operations are performed against `sys_regs[]`
- the response is transmitted with RS485 direction control on `PC3`

Hardware / Pin Mapping

The most relevant board-level signal definitions currently live in [functions/definitions_sensor.h]

Communications

- `UART1` TX: `PC0`
- `UART1` RX: `PC1`
- `RS485` DE/RE: `PC3`

DS18B20

- `DS18B20` #1: `PA2`
- `DS18B20` #2: `PA3`

Thermocouple / NTC shared digital lines

- shared sampled input: `PA5` (`MISO`-style bit input)
- shared clock output: `PA6`
- thermocouple chip select / enable: `PD1`
- NTC path enable: `PD7`

NTC mux select lines

- `A`: `PC2`
- `B`: `PF5`
- `C`: `PF4`

LEDs

- RX LED: `PD2`
- TX LED: `PD3`
- measure LED: `PD4`
- error LED: `PD5`
- run LED: `PD6`

File and Module Guide

Main application files

- [main.c](#)
Main control loop, system init, polling orchestration, and register update path.
- [DS.c](#)
Shared DS18B20 state machine and low-level 1-Wire bit-banging.
- [DS.h](#)
DS18B20 public types and APIs.

Custom firmware modules

The [functions/](#) folder contains the hand-maintained application logic:

- [functions/temp_sensors.c](#)
NTC acquisition, thermocouple acquisition, and derived temperature calculations.
- [functions/temp_sensors.h](#)
Sensor types, thermistor constants, and thermocouple state definitions.
- [functions/modbus.c](#)
Modbus RTU slave implementation and RS485 direction control.
- [functions/modbus.h](#)
Modbus public API.
- [functions/system_registers.h](#)
Live register map and control/status bitfields.
- [functions/system_registers.c](#)
Backing array for [sys_regs\[\]](#) and sensor enable helpers.
- [functions/definitions_sensor.h](#)
Board-level pin aliases, LEDs, and convenience macros.
- [functions/debug_uart2.c](#)
Auxiliary debug UART support.

Other files in [functions/](#) are alternate implementations, helpers, or retained references. Treat them as useful context, not necessarily active runtime code.

MCC-generated files

The [mcc_generated_files/](#) tree is generated by [MCC Melody](#) / MPLAB code generation. Important groups include:

- [system/](#)
- [timer/](#)

- [uart/](#)
- [spi/](#)

Do not assume manual changes in this tree are stable across regeneration. If you edit generated files directly, document why and expect the changes to be lost when [Sensor_Controller.mc3](#) is regenerated.

MCC project file

- [Sensor_Controller.mc3](#)

This is the MCC/Melody configuration file used by MPLAB. If clock, UART, timers, pins, or peripheral assignments change, this file should be kept in sync with the generated output.

Register Map

The active Modbus/register map is defined in [functions/system_registers.h](#).

Key register groups:

- [0x0000](#) firmware version
- [0x0001](#) / [0x0002](#) uptime low/high
- [0x0003](#) system status bitfield
- [0x0004](#) sensor enable bitfield
- [0x0005](#) to [0x000C](#) derived / priority temperatures
- [0x000D](#) to [0x0015](#) NTC temperatures and NTC error bits
- [0x0016](#) to [0x0018](#) K-type values and errors
- [0x0019](#) to [0x001B](#) DHT placeholders
- [0x001C](#) to [0x001F](#) DS18B20 values and errors

Important control bits:

- [STATUS_POLL_ACTIVE](#)
- [STATUS_SYSTEM_READY](#)
- [ENABLE_NTC1](#) through [ENABLE_NTC8](#)
- [ENABLE_KTYPE](#)
- [ENABLE_DS18B20_1](#)
- [ENABLE_DS18B20_2](#)

Build System

This is a generated MPLAB make project.

Relevant files:

- [Makefile](#)
- [nbproject/Makefile-impl.mk](#)
- [nbproject/Makefile-default.mk](#)
- [nbproject/Makefile-variables.mk](#)

Default artifact

The generated default production artifact path is:

- `dist/default/production/Sensor_Controller.X.production.hex`

Other useful outputs produced by a build include:

- `dist/default/production/Sensor_Controller.X.production.elf`
- `dist/default/production/Sensor_Controller.X.production.map`
- `dist/default/debug/Sensor_Controller.X.debug.elf`
- `dist/default/debug/Sensor_Controller.X.debug.map`

The `.hex` is the normal programming image for MPLAB/device programmers.

The `.elf` is the most useful file for:

- symbol-aware debugging
- source-level stepping
- converting to other output formats such as raw binary

The `.map` is useful when:

- checking memory placement
- inspecting code/data growth
- tracing which modules are linked in

Toolchain and IDE Requirements

To continue development on this project, a new developer should install:

1. `MPLAB X IDE`
2. `MPLAB XC8 Compiler` version `3.10` or a deliberately upgraded version after validation
3. `AVR-Dx Device Family Pack` version `2.7.321` or a validated equivalent
4. `MCC Melody` support matching the `.mc3` project
5. A supported programmer/debugger for `UPDI`

The project metadata currently expects `Atmel-ICE`, but a future developer can usually switch to another UPDI-capable tool in MPLAB if needed, such as:

- `MPLAB SNAP` if supported in the environment
- `PICKit 4 / MPLAB PICKit 4`
- another supported Microchip debugger with UPDI support

If a different tool is used, update the project properties and verify:

- communication interface remains `UPDI`
- target voltage is appropriate for the board
- erase/program options are still correct

Opening the Project in MPLAB X

1. Start `MPLAB X IDE`.

2. Use **File -> Open Project**.
3. Open the folder:
 - **Sensor_Controller.X**
4. Confirm that MPLAB loads the project as an embedded make project.
5. If prompted about missing packs, install the requested AVR pack.
6. If prompted about MCC-generated sources, allow MPLAB to index the project first before making changes.

Once opened, verify the active project properties:

- Device: **AVR128DB32**
- Compiler: **XC8**
- Configuration: **default**
- Programmer/debugger: **Atmel-ICE** or your selected UPDI tool

Building in MPLAB X

Production build

Use:

- **Run -> Clean and Build Main Project**

Expected main output:

- **dist/default/production/Sensor_Controller.X.production.hex**

Debug build

Use:

- **Debug -> Debug Main Project**

This will build the debug image and use the **.elf** for the debugger session.

Build from command line

From the project root, MPLAB-generated makefiles usually support:

```
make CONF=default
```

For a clean build:

```
make CONF=default clean
make CONF=default
```

If a developer uses command-line builds outside MPLAB, they must ensure:

- **XC8** is installed
- the MPLAB/XC8 tool binaries are on **PATH**, or MPLAB sets the environment
- the required AVR device pack is installed

Programming the AVR128DB32 from MPLAB X IDE

This is the primary recommended workflow for this project.

Hardware assumptions

The project is configured for:

- target device: **AVR128DB32**
- interface: **UPDI**
- tool: **Atmel-ICE**

The current project settings indicate:

- activation mode: **nohv**
- interface: **updi**
- communication speed: **0.500**
- erase before programming: **true**

Typical wiring expectations

A future developer should confirm the actual board wiring, but a normal UPDI programming setup requires:

- **UPDI**
- **GND**
- target **VCC** reference if the tool needs to sense target voltage

If the programmer does not power the board:

- the board must already be powered correctly before programming

If the programmer is configured to power the board:

- confirm the target voltage matches the hardware design before enabling it

The current project configuration has:

- **poweroptions.powerenable = false**

That means the IDE project is not currently set to power the board from the tool.

Program from the IDE

1. Connect the programmer/debugger to the board using **UPDI**.
2. Power the board correctly.
3. In MPLAB X, right-click the project.

4. Choose **Properties**.
5. Under hardware tools, select the connected programmer.
6. Confirm device is **AVR128DB32**.
7. Click **Apply**.
8. Build the project:
 - **Run -> Clean and Build Main Project**
9. Program the board:
 - **Run -> Program Main Project**

Expected behavior:

- MPLAB builds or uses the latest production artifact
- the tool erases the device
- the **.hex** is programmed into flash
- MPLAB reports program success in the output window

Debug from the IDE

1. Connect the UPDI tool.
2. Select the project as main project.
3. Choose:
 - **Debug -> Debug Main Project**
4. MPLAB will use the debug **.elf**
5. Breakpoints, stepping, variables, and symbol view should then be available

Use the debug path when working on:

- interrupt timing
- Modbus framing behavior
- DS18B20 state transitions
- register updates
- sensor conversion anomalies

Programming Using the Built Firmware File

If a developer does not want to press Program directly from the IDE, they can build the firmware and then load the generated image using a programmer utility.

Primary production file

Use this file for normal programming:

- **dist/default/production/Sensor_Controller.X.production.hex**

This is the standard file generated for the project and is the best file to treat as the official firmware image.

Why **.hex** is the normal file

Intel HEX contains:

- flash data
- addresses
- a text-based encoded representation understood by programmer tools

For AVR/MPLAB workflows, `.hex` is generally more natural than a flat `.bin`.

About `.bin` Files for This Project

This project does **not** currently generate a native `.bin` artifact by default.

Out of the box, it generates:

- production `.hex`
- production `.elf`
- debug `.elf`
- `.map`

If someone says “program the bin”, they usually mean one of two things:

1. They actually want to program the normal firmware image, in which case they should use the `.hex`.
2. Their external manufacturing or programming tool specifically requires a raw binary file, in which case a `.bin` must be created from the `.elf` or `.hex`.

Recommended rule

- Use the `.hex` whenever the tool supports it.
- Only use `.bin` if the external tool explicitly requires raw binary.

Creating a `.bin` from the Built Firmware

If a raw binary file is required, generate it from the production `.elf`.

Source file

Use:

- `dist/default/production/Sensor_Controller.X.production.elf`

Example conversion command

Using the AVR `objcopy` that ships with XC8:

```
avr-objcopy -O binary `
dist/default/production/Sensor_Controller.X.production.elf `
dist/default/production/Sensor_Controller.X.production.bin
```

If `avr-objcopy` is not on `PATH`, use the one from the XC8 installation `bin` directory.

Important warning about `.bin`

A raw `.bin` does not carry address metadata the same way `.hex` does.

That means whoever programs the `.bin` must know:

- the binary is intended to start at application flash base address `0x0000`
- the programmer tool must place it at the correct flash offset

If the programmer utility asks for:

- start address
- offset
- memory region

the operator must enter the correct flash base settings for the `AVR128DB32`.

If there is any uncertainty, do not use `.bin`; use `.hex` instead.

Programming a File Without Rebuilding in MPLAB X

If a developer already has a built file and wants to use MPLAB tools without rebuilding from the project, the usual approach is to use a device programming utility such as MPLAB IPE or an equivalent tool supported by the installed programmer.

Recommended file:

- `Sensor_Controller.X.production.hex`

Typical high-level procedure:

1. Open the programming utility.
2. Select device `AVR128DB32`.
3. Select the connected `UPDI` tool.
4. Browse to the `.hex` file.
5. Power the target correctly.
6. Program the device.
7. Verify if the tool supports verification.

If using a `.bin` instead:

- ensure the tool supports raw binary
- ensure the correct memory base address is configured

Safe Development Workflow

For someone continuing development on this firmware, the safest routine is:

1. Open the project in MPLAB X.
2. Build without changing code to confirm the environment is valid.
3. Program the current production image and confirm the hardware still responds over Modbus.
4. Only then begin making code changes.
5. After each change:

- build
- program
- verify sensor readings and Modbus access

This matters because the project mixes:

- generated peripheral code
- hand-written low-level bit-banged sensor interfaces
- live register and comms logic

Small timing changes can affect the hardware behavior.

Recommended Verification After Any Change

At minimum, verify:

1. The board still boots and the run LED behavior is normal.
2. The Modbus slave still answers on address 2.
3. Reads of:
 - system registers
 - NTC registers
 - K-type registers
 - DS18B20 registers
 are still valid.
4. RS485 direction switching still works reliably.
5. DS18B20 conversions still cycle correctly.
6. NTC values remain stable and reasonable.
7. Thermocouple faults still map to the expected register values.

Suggested Areas for Future Development

1. Normalize engineering-unit scaling

The current register table mixes x10 and x100 temperature formats. That makes master-side parsing fragile.

Recommended future work:

- choose one scaling convention for all temperature registers
- update the register map comments
- update master-side decoding to match

2. Review thermocouple calibration path

The current code applies:

- thermocouple scaling from raw MAX31855-style data
- an extra +35 offset

That should be reviewed against:

- actual hardware design
- cold-junction expectations
- required calibration behavior

3. Review K-type polling state logic

The K-type state machine currently becomes `READ_READY` and then remains readable every loop. That may be intentional, but it should be reviewed if the design really intended timed conversion windows rather than constant foreground reads.

4. Review Modbus exception handling

Some exception-response calls in the current Modbus code are commented out. Today that means some invalid requests are effectively ignored rather than answered with a formal Modbus exception.

5. Add explicit test procedure documentation

This project would benefit from a simple hardware validation checklist covering:

- known-good sensor inputs
- expected register values
- expected scaling
- expected error behavior when sensors are disconnected

6. Clarify which legacy files are still relevant

The tree contains several alternate or older sensor-related files. It would help future maintenance to explicitly identify:

- active files
- deprecated files
- scratch/test/reference files

MCC / Generated Code Guidance

If a future developer edits peripheral setup manually in generated files, they must expect those edits to be overwritten.

Preferred approach:

1. change the configuration in `Sensor_Controller.mc3`
2. regenerate using MCC
3. review the generated diffs carefully
4. re-test all timing-sensitive code

Be especially cautious with regeneration if the project depends on:

- exact timer periods
- interrupt registration
- UART timing

- pin directions and defaults

Practical “First Day” Handoff Plan for a New Developer

A developer taking over this project should do the following in order:

1. Open [FIRMWARE_FLOW.drawio](#) and understand the architecture at a block level.
2. Read:
 - [main.c](#)
 - [functions/temp_sensors.c](#)
 - [functions/modbus.c](#)
 - [DS.c](#)
 - [functions/system_registers.h](#)
3. Build the project unchanged.
4. Program the board from MPLAB IDE over UPDI.
5. Confirm Modbus register access with a known-good master.
6. Confirm each sensor class returns sane values.
7. Only then begin refactoring or feature work.

Current Artifact Summary

As of the current project layout, a developer should expect:

- Main production programming file:
 - [dist/default/production/Sensor_Controller.X.production.hex](#)
- Main debug file:
 - [dist/default/debug/Sensor_Controller.X.debug.elf](#)
- Main production symbol/debuggable file:
 - [dist/default/production/Sensor_Controller.X.production.elf](#)
- Optional raw binary if manually converted:
 - [dist/default/production/Sensor_Controller.X.production.bin](#)

Final Advice for Future Maintenance

The fastest way to damage this project is to treat it like a pure software-only codebase. It is not. This firmware is tightly coupled to:

- board wiring
- bit-level sensor timing
- UPDI programming flow
- RS485 bus behavior
- register map expectations on the external master

When making changes:

- change one thing at a time
- rebuild immediately
- reprogram immediately

- verify on hardware immediately

If a behavior looks “wrong” in firmware, always check whether it may actually be:

- scaling mismatch
- electrical/pin conflict
- sensor timing
- RS485 direction timing
- stale assumptions in the master-side parser